

CI-CD - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

CI/CD — MASTER NOTES	4
A Complete Reference Book for Interviews, Production Work & System Understanding	4
TOPIC 1: BUILD PIPELINES	4
1. Introduction	4
1.1 What is CI/CD, and why does it exist?	4
1.2 The exact problem it solves — Recap From the Git Section's Pull Requests Topic	4
2. What Is a "Build Pipeline"?	4
3. The Standard Pipeline Stages	4
4. Real Life Analogy	5
5. Frequently Asked Interview Questions	5
TOPIC 2: GITHUB ACTIONS	5
1. Introduction	5
2. FULL STEP-BY-STEP: A Complete CI/CD Pipeline for Your Spring Boot App	6
STEP 1 — Create the Workflow File	6
3. What Happens on a Pull Request (Tying Directly Back to the Git Section)	7
4. Real Life Analogy	7
5. Frequently Asked Interview Questions	7
TOPIC 3: JENKINS	7
1. Introduction	7
2. GitHub Actions vs Jenkins — Comparison Table	7
3. Code Example — A Jenkinsfile (Jenkins' Pipeline-as-Code)	8
4. Real Life Analogy	8
5. Frequently Asked Interview Questions	8
TOPIC 4: DEPLOYMENT PIPELINES	9
1. Introduction	9
2. The Standard Multi-Environment Flow	9

3. Deployment Strategies — Minimizing Risk/Downtime	9
4. Code Example — A Manual Approval Gate Before Production (GitHub Actions)	9
5. Real Life Analogy	9
6. Frequently Asked Interview Questions	10

TOPIC 5: ENVIRONMENT VARIABLES 10

1. Introduction	10
2. Code Example — Using Secrets in GitHub Actions	10
3. Setting Up Secrets (GitHub)	10
4. Real Life Analogy	11
5. Edge Cases	11
6. Frequently Asked Interview Questions	11

TOPIC 6: ARTIFACT MANAGEMENT 11

1. Introduction	11
2. Why This Matters — "Build Once, Deploy Many Times"	12
3. Common Artifact Types and Their Repositories	12
4. Code Example — Versioning and Publishing a Docker Image Artifact	12
5. Real Life Analogy	12
6. Common Mistakes	12
7. Frequently Asked Interview Questions	13
8. Revision Notes (Entire CI/CD Section Consolidated)	13
9. Homework (Covers Entire CI/CD Section)	13
FINAL SUMMARY — CI/CD COMPLETE	14

CI/CD — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

TOPIC 1: BUILD PIPELINES

1. Introduction

1.1 What is CI/CD, and why does it exist?

CI (Continuous Integration) means AUTOMATICALLY building and testing every code change the MOMENT it's pushed — catching bugs immediately, rather than discovering them days later when many changes have piled up. **CD (Continuous Delivery/Deployment)** extends this to AUTOMATICALLY package and DEPLOY that validated code — all the way to production, with little or no manual intervention.

1.2 The exact problem it solves — Recap From the Git Section's Pull Requests Topic

Recall from Git's Pull Requests topic: "Automated CI checks run (tests, linting, build)" the moment a PR is opened. CI/CD is the SYSTEM that actually PERFORMS this — WITHOUT it, every code change would require a HUMAN to manually run tests, manually build a JAR/Docker image, and manually copy it to a server — slow, inconsistent, and error-prone (forgetting a step, testing against the WRONG environment).

WITHOUT CI/CD:

Developer pushes code -> HUMAN manually runs `mvn test` -> HUMAN manually runs `mvn package` -> HUMAN manually builds a Docker image -> HUMAN manually SSHs into a server and deploys it -> easy to skip a step, forget to test, or deploy the WRONG version

WITH CI/CD:

Developer pushes code -> PIPELINE AUTOMATICALLY: runs tests -> builds -> packages into a Docker image -> pushes to a registry -> deploys to production - CONSISTENT, FAST, and NEVER accidentally skips a step

2. What Is a "Build Pipeline"?

A pipeline is a SEQUENCE of automated STAGES, each depending on the previous one succeeding — directly analogous to Maven's build lifecycle (Maven section) or a Docker multi-stage build (Docker section), just orchestrated at the level of your ENTIRE project (code → tests → package → deploy), not just within one tool.

3. The Standard Pipeline Stages

1. CHECKOUT -> pull the latest code from Git
2. BUILD -> compile the code (mvn compile / npm run build)
3. TEST -> run automated tests (mvn test / npm test)
4. PACKAGE -> produce a deployable artifact (a .jar, or a Docker image)
5. PUBLISH -> push the artifact to a registry (Nexus/Artifactory, or a Docker registry - AWS section's ECR)
6. DEPLOY -> release the artifact to an environment (staging, then production)

Critical principle: each stage should FAIL FAST — if TESTS fail, the pipeline STOPS immediately, and the artifact is NEVER packaged/deployed, exactly like Maven's build lifecycle stopping at `test` if a unit test fails, never reaching `package`.

4. Real Life Analogy

- **Restaurant:** A build pipeline is like a restaurant's STANDARDIZED quality-control assembly line for EVERY new recipe variation — ingredients are checked (checkout), the dish is prepared (build), a TASTE TEST is performed (test) — and if the taste test FAILS, the dish NEVER proceeds to being PLATED (packaged) or SERVED to customers (deployed), automatically and consistently, every single time, without a human needing to remember to enforce this manually.

5. Frequently Asked Interview Questions

- 1 **What do CI and CD stand for?** Continuous Integration (automatically building/testing every change) and Continuous Delivery/Deployment (automatically packaging and releasing validated code).
 - 2 **What problem does CI/CD solve compared to manual deployment?** Eliminates the inconsistency, slowness, and human error risk of manually running builds/tests/deployments for every single code change.
 - 3 **What are the standard stages of a build pipeline?** Checkout, Build, Test, Package, Publish, Deploy.
 - 4 **Why should a pipeline "fail fast" (e.g., stop immediately if tests fail)?** To prevent a broken/untested change from ever being packaged or deployed, catching problems as early and cheaply as possible in the pipeline.
-
-

TOPIC 2: GITHUB ACTIONS

1. Introduction

What is it? GitHub Actions is GitHub's BUILT-IN CI/CD platform — you define a pipeline as a YAML file LIVING directly inside your repository (`.github/workflows/`), and GitHub automatically RUNS it in response to events (a push, a pull request).

2. FULL STEP-BY-STEP: A Complete CI/CD Pipeline for Your Spring Boot App

STEP 1 — Create the Workflow File

```
# .github/workflows/ci-cd.yml
name: Build, Test, and Deploy

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest # GitHub provides a fresh virtual machine for EVERY run

    steps:
      - name: Checkout code
        uses: actions/checkout@v4 # a REUSABLE, pre-built "action" - checks out your repo

      - name: Set up Java 17
        uses: actions/setup-java@v4
        with:
          java-version: "17"
          distribution: "temurin"

      - name: Run tests (Maven section!)
        run: mvn test

      - name: Build the JAR
        run: mvn clean package -DskipTests

      - name: Build Docker image (Docker section!)
        run: docker build -t my-company/employee-api:${{ github.sha }} .

      - name: Log in to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME } # SECRETS - Topic 5!
          password: ${ secrets.DOCKERHUB_TOKEN }

      - name: Push Docker image
        run: docker push my-company/employee-api:${{ github.sha }}

    deploy:
      needs: build-and-test # ONLY runs if 'build-and-test' SUCCEEDED - the "fail fast" principle!
      runs-on: ubuntu-latest
      if: github.ref == 'refs/heads/main' # ONLY deploy from the main branch, not every PR
      steps:
        - name: Deploy to AWS ECS (AWS section!)
          run: |
            aws ecs update-service --cluster my-cluster --service employee-api-service --force-new-deployment
```

3. What Happens on a Pull Request (Tying Directly Back to the Git Section)

Developer opens a PR -> GitHub Actions AUTOMATICALLY triggers the 'build-and-test' job -> if tests FAIL, GitHub shows a RED "X" directly on the PR, and (with branch protection rules, Git section Topic 10) BLOCKS merging until fixed -> if tests PASS, a GREEN checkmark appears, and the PR can be merged

4. Real Life Analogy

- **Restaurant:** GitHub Actions is like a restaurant chain's AUTOMATED quality-control STATION built directly into their central order-management system — the MOMENT a new recipe variation is submitted, the station automatically runs its standard checks, without a human needing to manually walk it over to inspection.

5. Frequently Asked Interview Questions

- 1 **What is GitHub Actions?** GitHub's built-in CI/CD platform, running YAML-defined pipelines ("workflows") in response to repository events like pushes and pull requests.
- 2 **What does `needs: build-and-test` do in a GitHub Actions job?** Makes that job depend on `build-and-test` succeeding first — it only runs if the prior job completed successfully.
- 3 **How does GitHub Actions connect to branch protection rules (from the Git section)?** A failing CI check can be configured as a REQUIRED status check, blocking a PR from being merged until it passes.

TOPIC 3: JENKINS

1. Introduction

What is it? Jenkins is a widely-used, SELF-HOSTED, open-source automation server — an OLDER, more traditional alternative to GitHub Actions, still extremely common in large enterprises (especially those with on-premises infrastructure or complex, highly-customized pipeline needs).

2. GitHub Actions vs Jenkins — Comparison Table

Aspect	GitHub Actions	Jenkins
Hosting	Managed BY GitHub (no server to maintain)	SELF-HOSTED — you run and maintain your OWN Jenkins server
Setup complexity	Low (YAML file in your repo)	Higher (install/configure/maintain the server itself)
Flexibility/plugins	Growing marketplace of "actions"	MASSIVE, mature plugin ecosystem (many years older)
Common in	Modern, cloud-native teams already on GitHub	Larger, established enterprises, complex legacy pipeline needs

3. Code Example — A `Jenkinsfile` (Jenkins' Pipeline-as-Code)

```
// Jenkinsfile
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Test') {
      steps { sh 'mvn test' }
    }
    stage('Build') {
      steps { sh 'mvn clean package -DskipTests' }
    }
    stage('Build & Push Docker Image') {
      steps {
        sh 'docker build -t my-company/employee-api:${BUILD_NUMBER} .'
        sh 'docker push my-company/employee-api:${BUILD_NUMBER}'
      }
    }
    stage('Deploy') {
      when { branch 'main' }
      steps {
        sh 'aws ecs update-service --cluster my-cluster --service employee-api-service --force-new-deployment'
      }
    }
  }
}
```

Notice the STAGES here are conceptually IDENTICAL to the GitHub Actions workflow (Topic 2) — CI/CD TOOLS differ in syntax/hosting, but the underlying PIPELINE CONCEPT (Topic 1) is universal.

4. Real Life Analogy

- **Restaurant:** GitHub Actions is like using a modern, THIRD-PARTY-MANAGED kitchen-automation SERVICE that a restaurant simply subscribes to. Jenkins is like the restaurant BUILDING AND OPERATING its own custom, in-house automation system — more work to set up and maintain, but with MAXIMUM flexibility to customize EVERY detail exactly to that specific restaurant's unique needs.

5. Frequently Asked Interview Questions

- 1 **What is Jenkins?** A self-hosted, open-source automation server for building CI/CD pipelines, an older and more traditional alternative to managed platforms like GitHub Actions.
- 2 **What's the main trade-off between GitHub Actions and Jenkins?** GitHub Actions is managed (less setup/maintenance) but tied to GitHub; Jenkins is self-hosted (more setup/maintenance burden) but offers maximum flexibility via its mature plugin ecosystem.
- 3 **What is a `Jenkinsfile`?** Jenkins' pipeline-as-code file (typically Groovy-based), defining the pipeline's stages directly in version control, conceptually equivalent to a GitHub Actions workflow YAML file.

TOPIC 4: DEPLOYMENT PIPELINES

1. Introduction

What is it? A deployment pipeline extends the BUILD pipeline (Topic 1) with the STAGES specifically responsible for **RELEASING** validated code to **REAL** environments — typically progressing through **MULTIPLE** environments (Dev → Staging → Production) with increasing levels of confidence/testing before reaching real users.

2. The Standard Multi-Environment Flow

```
DEVELOPMENT STAGING PRODUCTION
(auto-deployed on (auto-deployed on (deployed MANUALLY, or
every merge to every merge to main, automatically ONLY after
a feature branch) after passing ALL tests) staging verification passes)
| | |
v v v
Developers test QA / automated REAL USERS - maximum
quickly, informally end-to-end tests caution, often with a
run here MANUAL approval gate
```

3. Deployment Strategies — Minimizing Risk/Downtime

Strategy	How It Works
Rolling Deployment	Replace old instances with new ones a FEW AT A TIME (recall ECS/Kubernetes rolling updates, Microservices/AWS sections) — never a full outage
Blue-Green Deployment	Run TWO complete, identical production environments ("Blue" = current live, "Green" = new version) — switch ALL traffic to "Green" INSTANTLY once verified, with "Blue" kept ready as an IMMEDIATE rollback option
Canary Deployment	Route a SMALL PERCENTAGE of real traffic (e.g., 5%) to the new version first, monitoring closely, before gradually increasing to 100%

4. Code Example — A Manual Approval Gate Before Production (GitHub Actions)

```
deploy-production:
needs: deploy-staging
runs-on: ubuntu-latest
environment:
name: production # configured in GitHub to REQUIRE a manual approval before this job runs
steps:
- name: Deploy to Production
run: aws ecs update-service --cluster prod-cluster --service employee-api-service
--force-new-deployment
```

5. Real Life Analogy

- **Restaurant:** A deployment pipeline is like a NEW recipe going through progressively more serious testing stages — first tried informally by the CHEFS THEMSELVES (dev), then served to a SMALL group of trusted regular customers for feedback (staging/canary), and ONLY THEN rolled out to the FULL, general public menu (production) — with the OLD recipe kept ready to instantly bring back if something goes wrong (blue-green/rollback).

6. Frequently Asked Interview Questions

- 1 **What's the typical progression of environments in a deployment pipeline?** Development → Staging → Production, with increasing levels of testing/confidence required at each stage.
- 2 **What is Blue-Green deployment?** Running two complete, identical production environments, switching ALL traffic instantly to the new one once verified, keeping the old one ready as an immediate rollback option.
- 3 **What is a Canary deployment?** Gradually routing an increasing percentage of real traffic to a new version, starting small, to catch problems before a full rollout.
- 4 **Why would you add a manual approval gate before deploying to production?** To have a human explicitly confirm a deployment should proceed to real users, even after all automated checks pass — a deliberate safety checkpoint for the highest-risk environment.

TOPIC 5: ENVIRONMENT VARIABLES

1. Introduction

What is it? Environment variables let you inject CONFIGURATION (database URLs, API keys) and SECRETS (passwords, tokens) into your pipeline/application WITHOUT hardcoding them in source code — directly recapping the SAME principle from Spring Boot's Configuration Properties topic and Docker's "never bake secrets into images" guidance, now applied specifically to the CI/CD PIPELINE itself.

2. Code Example — Using Secrets in GitHub Actions

```
steps:
- name: Deploy with database credentials
  env:
    DB_PASSWORD: ${ secrets.DB_PASSWORD } # pulled from GitHub's ENCRYPTED secrets store, NEVER
  run: | # visible in logs or the repository itself
    echo "Deploying with configured database..."
    ./deploy.sh
```

3. Setting Up Secrets (GitHub)

- 1 Go to your repository → **Settings** → **Secrets and variables** → **Actions**.
- 2 Click **New repository secret**.

- 3 Add the name (`DB_PASSWORD`) and value — GitHub ENCRYPTS this and NEVER displays it again, even to repository admins; it's only injected into workflow runs as `secrets.DB_PASSWORD`.

4. Real Life Analogy

- **Bank:** Environment variables/secrets are like a bank's SEALED, need-to-know instruction envelope handed DIRECTLY to a specific authorized teller AT THE MOMENT of a transaction — never written down publicly on a shared bulletin board (committed to source code) where ANYONE could read it.

5. Edge Cases

Case	Result
Accidentally echo-ing a secret value directly in a pipeline script	Most CI/CD platforms automatically MASK known secret values in log output, but this is NOT a 100% reliable safety net — never deliberately print secrets
Committing a secret directly into a config file in Git	A SERIOUS incident — the secret is now in Git HISTORY forever (recall the Git section's "leaked secret" scenario) and must be ROTATED/invalidated immediately, not just deleted from the latest commit

6. Frequently Asked Interview Questions

- 1 **Why should secrets never be hardcoded directly in a pipeline's source file?** They'd be visible to anyone with repository access, and permanently preserved in Git history even if later "removed" — secrets should be stored in the CI/CD platform's dedicated, encrypted secrets management feature instead.
- 2 **How are secrets typically referenced in a GitHub Actions workflow?** Via `${{ secrets.SECRET_NAME }}`, pulling from GitHub's encrypted secrets store, configured in repository settings.
- 3 **What must you do if a secret is accidentally committed to a Git repository?** ROTATE (invalidate and replace) that secret immediately — simply removing it from the latest commit is NOT sufficient, since it remains in the repository's history.

TOPIC 6: ARTIFACT MANAGEMENT

1. Introduction

What is it? An artifact is the actual BUILT, DEPLOYABLE OUTPUT of your pipeline (a `.jar` file, a Docker image, a compiled frontend bundle) — Artifact Management is the practice of STORING, VERSIONING, and RETRIEVING these build outputs reliably, so the EXACT SAME artifact that passed testing is what actually gets deployed (never a "close enough" rebuild).

2. Why This Matters — "Build Once, Deploy Many Times"

BAD PRACTICE: rebuild the application SEPARATELY for staging AND production
-> risk of SUBTLE differences between the two builds (different dependency resolution, different timestamps) - what you TESTED isn't EXACTLY what you DEPLOYED!

GOOD PRACTICE ("build once"): build the artifact EXACTLY ONCE, store it in an artifact repository, and DEPLOY that SAME, IDENTICAL, already-tested artifact to EVERY environment - guaranteeing what passed testing is EXACTLY what reaches production

3. Common Artifact Types and Their Repositories

Artifact Type	Repository
Maven <code>.jar/.war</code>	Nexus Repository, JFrog Artifactory
Docker images	Docker Hub, AWS ECR (AWS section), Google Artifact Registry
npm packages (React frontend builds)	npm registry, GitHub Packages

4. Code Example — Versioning and Publishing a Docker Image Artifact

```
- name: Build Docker image with a VERSIONED tag
run: |
  docker build -t my-company/employee-api:${{ github.sha }} . # tag by GIT COMMIT SHA - unique, traceable
  docker tag my-company/employee-api:${{ github.sha }} my-company/employee-api:latest

- name: Push BOTH tags
run: |
  docker push my-company/employee-api:${{ github.sha }}
  docker push my-company/employee-api:latest
```

Tagging by the GIT COMMIT SHA (rather than JUST `latest`, recall Docker section's warning about `latest` being a "moving target") gives you an EXACT, TRACEABLE link between a specific deployed artifact and the EXACT source code that produced it — essential for debugging "which version is actually running in production right now?"

5. Real Life Analogy

- **Library:** Artifact management is like a library maintaining ONE definitive, CATALOGED, VERSION-STAMPED copy of a published book EDITION — every branch/reader accessing "Edition 3, printed batch #47" is GUARANTEED to be reading the EXACT SAME content, rather than each branch printing its OWN separate copy that MIGHT subtly differ.

6. Common Mistakes

- Rebuilding the application separately for each environment instead of building ONCE and promoting the SAME artifact through environments — risks subtle, hard-to-diagnose differences between what was tested and what's deployed.

- Only ever tagging Docker images as `latest`, losing the ability to trace exactly which version is running in production or roll back to a specific known-good version.

7. Frequently Asked Interview Questions

- 1 **What is a build artifact?** The actual built, deployable output of a pipeline — a `.jar` file, Docker image, or compiled frontend bundle.
- 2 **What does "build once, deploy many times" mean, and why does it matter?** Building an artifact EXACTLY once and promoting that SAME artifact through every environment (staging, production), rather than rebuilding separately for each — guaranteeing what was tested is EXACTLY what gets deployed.
- 3 **Why tag Docker image artifacts by Git commit SHA instead of only `latest`?** It provides an exact, traceable link between a specific deployed artifact and the exact source code version that produced it, essential for debugging and enabling precise rollbacks.

8. Revision Notes (Entire CI/CD Section Consolidated)

CI/CD — CHEAT SHEET

=====

SOLVES: manual build/test/deploy inconsistency and human error - AUTOMATES the ENTIRE pipeline from code push to production

BUILD PIPELINE STAGES: Checkout -> Build -> Test -> Package -> Publish -> Deploy - FAIL FAST (stop immediately on any failure, never deploy broken code)

GITHUB ACTIONS: YAML workflows in `.github/workflows/`, triggered by push/PR, jobs can `needs:` each other, ties DIRECTLY to Git branch protection (failing checks block merge)

JENKINS: self-hosted alternative, Jenkinsfile (Groovy pipeline-as-code), SAME stage concept, more setup/maintenance but maximum flexibility

DEPLOYMENT PIPELINES: Dev -> Staging -> Production, increasing confidence
ROLLING (few instances at a time) / BLUE-GREEN (instant full switch + instant rollback) / CANARY (gradual traffic percentage) deployment strategies
manual approval gates before production are common/recommended

ENVIRONMENT VARIABLES/SECRETS: NEVER hardcode in source - use the CI/CD platform's encrypted secrets store (`${{ secrets.X }}` in GitHub Actions)
leaked secret -> ROTATE immediately, deleting the commit is NOT enough

ARTIFACT MANAGEMENT: "build once, deploy many times" - same tested artifact promoted through EVERY environment, never rebuilt per-environment
tag by GIT COMMIT SHA (traceable) not just "latest" (a moving target)
Nexus/Artifactory (jars), ECR/Docker Hub (images), npm registry (frontend)

9. Homework (Covers Entire CI/CD Section)

- 1 Write a GitHub Actions workflow for a small Spring Boot project: checkout, run tests, build the jar, and build a Docker image tagged with the commit SHA.
- 2 Add a `secrets`-based database password to your workflow, verifying it never appears in plain text in the logs.
- 3 Extend your workflow with a `deploy` job that `needs:` the build/test job and only runs on the `main` branch.

- 4 Research and write 3-4 sentences comparing rolling, blue-green, and canary deployment strategies, and when you'd choose each.
-

FINAL SUMMARY — CI/CD COMPLETE

This concludes the **CI/CD** section — 6 topics:

- 1 **Build Pipelines** — the universal stage concept, fail-fast principle.
- 2 **GitHub Actions** — a full, working Spring Boot + Docker + AWS pipeline.
- 3 **Jenkins** — the self-hosted alternative, `Jenkinsfile`.
- 4 **Deployment Pipelines** — multi-environment flow, rolling/blue-green/canary strategies.
- 5 **Environment Variables** — secrets management, never hardcoding, rotation on leaks.
- 6 **Artifact Management** — "build once, deploy many times," traceable versioning.

This is the automation layer tying together EVERYTHING from this entire course — Git, Maven, Docker, AWS, and your Spring Boot/React application — into a single, reliable, automated path from a code commit to running production software. Next: Linux, covering the operating system underlying most of the servers/containers this pipeline actually deploys to.

(END OF CI/CD MASTER NOTES — ready to proceed to Linux whenever you say "Next Topic.")